

KỊCH BẢN THUYẾT TRÌNH — CHUYÊN ĐỀ RNN (bộ beamer 26 slide)

Bộ slide: `rnn_slides.tex` (26 slide, XeLaTeX beamer) · tổng ~25–30 phút, 4 người trình bày

Nhóm: Văn Thị Mai Linh · Nguyễn Văn Minh Lực · Giáp Minh Hiếu · Nguyễn Nam Hải
— GVHD: TS. Trần Đình Quế

Nguyên tắc: nói công thức bằng LỜI trước, chỉ vào slide sau; mỗi slide 1 ý chính; số liệu đọc đúng như trên slide, không chế biến.

GỢI Ý PHÂN CÔNG (4 người — đối tự do)

Phần	Thời lượng
Người 1 — Mở đầu & động cơ (slide 1–9)	~7 phút
Người 2 — Lỗi kiến trúc RNN (slide 10–16)	~8,5 phút
Người 3 — Huấn luyện & điểm nghẽn (slide 17–20)	~5 phút
Người 4 — Thực nghiệm & tiến hoá (slide 21–26)	~6,5 phút

MỞ ĐẦU (~20 giây — trước slide 1)

Chào thầy và các bạn. Nhóm em trình bày chuyên đề Mạng nơ-ron hồi quy — đi từ động cơ vì sao MLP và 1D-CNN chưa đủ cho dữ liệu chuỗi, vào lỗi kiến trúc RNN với trạng thái ẩn và ba ma trận W U V , phần huấn luyện BPTT cùng hai chứng gradient, và cuối cùng là thực nghiệm trên giá vàng bạc, Amazon cùng hướng tiến hóa LSTM, GRU, Transformer.

SLIDE 1 — Trang bìa (~20 giây)

Chào thầy và các bạn. Nhóm em trình bày chuyên đề Học Sâu về Mạng nơ-ron hồi quy RNN — từ hạn chế của MLP và 1D-CNN đến nguyên lý xử lý dữ liệu chuỗi có trạng thái ẩn. Em xin phép giới thiệu các thành viên thực hiện.

SLIDE 2 — Bản chất dữ liệu chuỗi (~60 giây)

Dữ liệu chuỗi là dữ liệu mà THỨ TỰ xuất hiện quyết định hoàn toàn ngữ nghĩa. Tại thời điểm t , điểm dữ liệu luôn tương quan mật thiết với các điểm quá khứ t trừ 1, t trừ 2, v.v.

Hai dạng chính: token rời rạc như văn bản, DNA; và chuỗi số liên tục như giá vàng, cổ phiếu, nhịp tim. Ví dụ bên trái: “Hà Nội không mưa” đảo thành “Mưa không Hà Nội” là vô nghĩa — thứ tự tạo ngữ nghĩa. Bên phải: cùng bốn con số 82.5, 83.1, 84.0, 85.2 xếp tăng thì là xu hướng tăng, xếp ngược là giảm.

SLIDE 3 — Bài toán dự đoán phần tử kế tiếp (~45 giây)

Mục tiêu chung của cả chuyên đề: cho chuỗi quá khứ $x-1$ đến $x-t$, dự đoán giá trị tương lai $x-t$ cộng 1. Hai hiện thân tiêu biểu: bên NLP là next word prediction — “We are learning AI” thì từ kế tiếp kỳ vọng là “is”; bên chuỗi thời gian là dự báo giá — cửa sổ 30 ngày lịch sử dự đoán biến động ngày 31.

SLIDE 4 — Hai pipeline tiền xử lý (~60 giây)

Dữ liệu thô không đưa trực tiếp vào mạng được. Với văn bản: tokenizer tách từ, ánh xạ vocab index, padding cho đều độ dài, rồi embedding ra tensor Batch nhân Seq-Len nhân d. Với chuỗi số: làm sạch, chia train 70 val 15 test 15, scaling — ghi nhớ chỉ fit trên tập train — log1p rồi MinMax, cửa sổ trượt 30 ngày ra tensor N nhân 30 nhân 1. Hai pipeline này là nền cho mọi thực nghiệm phía sau.

SLIDE 5 — MLP & cơ chế ghép phẳng (~45 giây)

Cách tiếp cận đầu tiên: mạng truyền thẳng MLP. Nó bắt buộc biến toàn bộ ngữ cảnh thành MỘT vector phẳng: mỗi token thành một vector, concatenate lại thành vector 1 nhân 20, rồi qua các tầng Linear. Chỉ vào sơ đồ: trục thời gian đã biến mất hoàn toàn ngay từ phép ghép phẳng.

SLIDE 6 — Ba điểm nghẽn của MLP (~60 giây)

Thứ nhất, cửa sổ cố định: ma trận W gắn cứng với kích thước L — câu ngắn phải nhồi padding, câu dài hơn L bị cắt cụt. Thứ hai, mất tính tuần tự: flatten xóa sạch trục thời gian, mạng xem từ ở vị trí 1 và vị trí 5 là hai tọa độ độc lập. Thứ ba, bùng nổ tham số: số trọng số tầng đầu bằng L nhân d nhân số nơ-ron — ngữ cảnh tăng gấp đôi, trọng số tăng gấp đôi, dễ quá khớp.

SLIDE 7 — Bước tiến 1D-CNN (~45 giây)

1D-CNN khắc phục việc gắn cứng vị trí bằng kernel TRƯỢT: kernel kích thước k bằng 2 quét từng cặp từ kế nhau — bigram — cho ra feature map $c-1$ đến $c-4$. Chỉ riêng việc dùng lại một kernel ở mọi vị trí đã là bước tiến so với MLP. Rồi max-pooling lấy giá trị nổi bật nhất.

SLIDE 8 — Vì sao 1D-CNN vẫn chưa tối ưu (~45 giây)

Hai vấn đề còn lại. Một, tầm nhìn bó hẹp: kernel k bằng 2 chỉ bắt cặp từ đứng cạnh, không liên kết được chủ ngữ đầu câu với vị ngữ cuối đoạn. Hai, nặng hơn: max-pooling LẤY max nhưng QUÊN vị trí — 1.81 xuất hiện ở đầu hay cuối câu đều như nhau. Sửa một nửa bài toán thứ tự rồi tự tay xóa nửa còn lại.

SLIDE 9 — Bảng so sánh MLP / 1D-CNN / RNN (~45 giây)

Tổng kết động cơ bằng bảng: độ dài ngữ cảnh — MLP cố định cứng, CNN trượt theo k , RNN co giãn linh hoạt. Bảo toàn thứ tự — flatten mất sạch, CNN giữ cục bộ, RNN

duy trì toàn vẹn. Chia sẻ trọng số — MLP không chia sẻ, CNN theo không gian, RNN xuyên suốt thời gian. Bộ nhớ — chỉ RNN có trạng thái ẩn trực tiếp. Tham số — chỉ RNN hằng số theo độ dài. Từ đây chuyển sang phần lõi: RNN.

SLIDE 10 — Ý tưởng cốt lõi: trạng thái ẩn (~75 giây)

Trực giác: con người không bắt đầu tư duy lại từ đầu ở từng chữ — ta duy trì một dòng ý thức nén dần quá khứ. RNN làm y hết qua vector trạng thái ẩn h-t. Chỉ vào sơ đồ: ba ô RNN là CÙNG MỘT Ô lặp lại — đọc “Tôi” ra h-1 chỉ nhớ “Tôi”; “đang” cộng vào ra h-2 nhớ ngữ cảnh hai từ; “học” cho h-3 nhớ trọn câu.

Cột phải là bản chất toán: phương trình đệ quy h-t bằng f-theta của h-t trừ 1 và x-t — mỗi bước chỉ cần trạng thái liền trước và đầu vào hiện tại, nhưng h-t trừ 1 đã chứa cả quá khứ. Ba tính chất đáng nhớ: tóm tắt lịch sử VÔ HẠN trong vector cố định d-h chiều; cập nhật từng bước O của 1 — streaming, không lưu toàn chuỗi; và khác MLP cửa sổ — không phụ thuộc độ dài.

SLIDE 11 — Công thức forward & bộ ba W, U, V (~90 giây)

Ba phương trình, đọc bằng lời: a-t bằng W nhân x-t cộng U nhân h-t trừ 1 cộng b-h — tổng nhập ghép tin mới với trí nhớ cũ; h-t bằng tanh của a-t — trạng thái ẩn mới truyền sang bước kế; y-hat-t bằng V nhân h-t cộng b-y — đọc bộ nhớ ra dự đoán.

Bảng trái là kích thước: W thuộc R mũ d-h nhân d-x, input sang hidden, đọc tin mới; U thuộc R mũ d-h nhân d-h, hidden sang hidden, giữ trí nhớ — trái tim của RNN; V thuộc R mũ d-y nhân d-h đọc ra kết quả; cùng hai bias. Sơ đồ phải tóm lại một bước.

Hai câu hỏi hay bị hỏi: Vì sao tanh? — tâm 0, chặn trừ 1 cộng 1 giữ chuẩn h-t ổn định, đạo hàm một trừ tanh bình phương gọn; sigmoid lệch dương và đạo hàm tối đa 0.25 — chỉ bằng một phần tư của tanh — làm gradient tắt nhanh hơn hẳn; ReLU không chặn trên dễ bùng nổ trạng thái. Tên PyTorch: W là weight_ih_l0, U là weight_hh_l0 — Keras gọi kernel và recurrent. Cuối cùng: loss tổng theo thời gian bằng tổng ell của y-hat-t và y-t — MSE cho hồi quy.

SLIDE 12 — Ví dụ tính forward từng bước (~90 giây)

Để không tưởng tượng công thức suông, em tính tay với ma trận nhỏ: d-x bằng 2, d-h bằng 2, d-y bằng 1. W ma trận 0.5 0.2 trừ 0.1 0.4; U là 0.3 0.0 0.2 0.5; V là 1.0 trừ 1.0; x-1 là 1 0; h-0 bằng 0; bias đặt 0 và 0.1.

Bốn dòng bảng là bốn phép: một — W nhân x-1 cho 0.500 trừ 0.100, đọc tin mới; hai — U nhân h-0 bằng 0 vì chưa có kỷ ức, trang trắng; ba — cộng lại, nén tanh, được h-1 bằng 0.462 trừ 0.100; bốn — V nhân h-1 cộng bias ra y-hat-1 bằng 0.662.

Bước t bằng 2 lặp nguyên hệ thống: thay $x-2$ bằng 0 1, lấy $h-1$ vừa tính làm trí nhớ cũ — lần đầu tiên U nhân $h-1$ khác 0, thành phần hồi quy xuất hiện. Chạy dần đến T .
Dòng cuối là shape flow: x thuộc $R\ T$ nhân $d-x$, sang h thuộc $R\ T$ nhân $d-h$, rồi y thuộc $R\ T$ nhân $d-y$. PyTorch nn.RNN 2 2 cộng Linear 2 1 nạp đúng bộ số này — đặt bias đúng — chạy ra đúng kết quả.

SLIDE 13 — Nguyên lý chia sẻ trọng số (~75 giây)

Chuỗi 5 bước hay 5.000 bước, RNN đều dùng chung MỘT bộ W, U, V — bảng trên: cùng công thức lặp từ t bằng 1 tới T .

Hệ quả định lượng, khối trái: tổng tham số P bằng $d-h$ nhân $d-x$ cộng $d-h$ bình phương cộng $d-y$ nhân $d-h$ cộng $d-h$ cộng $d-y$ — KHÔNG chứa T . Cụ thể $d-x$ 2, $d-h$ 64, $d-y$ 1: 128 cộng 4096 cộng 64 cộng 64 cộng 1 bằng 4353 tham số cho mọi độ dài chuỗi.

Đối chiếu MLP cửa sổ L bằng 30: trọng số tầng đầu 30 nhân 2 nhân 64 bằng 3840 cộng 64 bias — chỉ để dùng cho MỘT độ rộng cửa sổ; còn W của RNN chỉ 128 nhưng tái sử dụng 30 lần. Bốn lợi ích: tham số hằng số theo T ; tổng quát hóa — mẫu hình ở đâu trong chuỗi cũng nhận diện như nhau; tự điều chuẩn — ít tham số trên cùng dữ liệu, kháng quá khớp; và analogy — kernel CNN chia sẻ theo KHÔNG GIAN thì RNN chia sẻ theo THỜI GIAN.

SLIDE 14 — Đồ thị tính toán mở cuộn (~75 giây)

Một ô RNN hồi quy khi mở cuộn trở thành mạng truyền thẳng CỰC SÂU, độ sâu đúng bằng T . Bên trái dạng cuộn với vòng U tự trở; bên phải mở ra: $h-0$ bằng 0, $x-1\ x-2\ x-3$ chấm chấm $x-T$ ở dưới; các ô $h-1$ đến $h-T$ cùng màu ở giữa — CÙNG MỘT Ô dùng chung W, U, V ; $y-1$ đến $y-T$ ở trên.

Ba điều rút ra: một — mở cuộn chính là mạng sâu T tầng, nên BPTT ở slide 17 không gì khác hơn backpropagation chạy trên đúng đồ thị này. Hai — cạnh ngang U là ĐƯỜNG TRÍ NHỚ, cắt nó là mất hết quá khứ. Ba — T giãn mãi tham số vẫn cố định, đồ thị dài thêm không thêm trọng số.

SLIDE 15 — Các dạng kiến trúc, phần 1 (~50 giây)

Tùy số bước vào ra, một ô RNN ghép thành các kiến trúc: One-to-One — 1 vào 1 ra, phân loại một ảnh, chỉ là mốc đối chiếu. One-to-Many — 1 vào chuỗi ra: sinh chú thích ảnh, sinh nhạc. Many-to-One — chuỗi vào 1 ra: phân loại cảm xúc review, và DỰ BÁO GIÁ NGÀY MAI — chính là dạng nhóm em dùng trong phần thực nghiệm cuối bài.

SLIDE 16 — Các dạng kiến trúc, phần 2 (~60 giây)

Many-to-Many đồng bộ: mỗi bước vào ra đúng một nhãn — gán nhãn từ loại POS, NER. Seq2Seq Encoder-Decoder: encoder nén chuỗi vào vector ngữ cảnh c , decoder sinh chuỗi ra với độ dài KHÁC đầu vào — dịch máy, tóm tắt. Bidirectional: hai chiều tiến lùi ghép h-t bằng vector hai nửa — mạnh cho NER, embedding; nhưng KHÔNG dùng dự báo tương lai vì chiều ngược nhìn được tương lai — rò rỉ dữ liệu. Chốt: bài thực nghiệm của nhóm là Many-to-One.

SLIDE 17 — BPTT (~90 giây)

Huấn luyện: tính sai số tại mọi thời điểm, lan truyền ngược gradient NGƯỢC dòng thời gian từ T về 1 — các mũi tên đỏ đứt trên sơ đồ. Forward chạy trái sang phải, gradient chảy phải sang trái qua từng h .

Công thức gradient cho U , đọc chậm: đạo hàm L theo U bằng tổng kép — với mỗi thời điểm t , cộng dồn mọi đường ngược về k nhỏ hơn t ; mỗi đường là một TÍCH các nhân tử đạo hàm $h-j$ theo $h-j$ trừ 1 nhân vào đạo hàm $h-k$ theo U . Chính tích lặp đó là gốc mọi khó khăn — sang slide sau.

SLIDE 18 — Triệt tiêu gradient (~75 giây)

Bản chất: đạo hàm L theo $h-k$ bằng đạo hàm L theo $h-t$ nhân chuỗi T trừ k khối đường-chéo của 1 trừ h bình phương nhân U . Đạo hàm tanh nằm trong khoảng 0 đến 1 — mỗi bước lùi gradient bị nhân thêm một hệ số nhỏ hơn 1 .

Khi trị riêng lớn nhất của U nhỏ hơn 1 : chuẩn U mũ T trừ k tiến về 0 theo hàm mũ khi chuỗi dài ra — tín hiệu ở các bước đầu gần như mất trắng, trọng số không cập nhật được để học liên kết quá khứ xa. Chuỗi càng dài, chứng quên càng nặng.

SLIDE 19 — Nổ gradient & clipping (~60 giây)

Ngược lại, trị riêng lớn hơn 1 : chuỗi nhân bùng nổ vô tận — bước nhảy trọng số quá lớn phá cực tiểu, tràn số NaN Inf.

Giải pháp đơn giản mà hiệu quả: cắt chuẩn gradient — g-clipped bằng g nhân θ chia chuẩn g khi chuẩn vượt ngưỡng θ . Vector quá dài bị co về đúng θ nhưng GIỮ NGUYÊN HƯỚNG. Lưu ý: clipping trị được BÌNH NỔ, không trị được TRIỆT TIÊU — vì gradient đã gần 0 thì không có gì để kéo giãn lại — cái sau phải chữa bằng kiến trúc LSTM, GRU.

SLIDE 20 — Tổng kết điểm nghẽn Simple RNN (~50 giây)

Ba điểm nghẽn của RNN thuần: một — không cơ chế chọn lọc, mọi thời điểm nén qua một hàm tanh duy nhất, thông tin quan trọng dễ bị rác làm loãng. Hai — mất trí nhớ dài hạn: do triệt tiêu gradient, thực tế chỉ nhớ 5 đến 10 bước gần nhất. Ba — bắt

buộc tính tuần tự: h-t chỉ tính được khi có h-t trừ 1, không thể song song hóa trên GPU như CNN hay Transformer.

SLIDE 21 — Thiết lập thực nghiệm (~50 giây)

Kiểm chứng trên hai tập Kaggle: giá vàng và bạc hàng ngày 1968 đến 2021; giá và khối lượng cổ phiếu Amazon 1997 đến 2023. Cấu hình: cửa sổ trượt 30 ngày dự đoán bước kế tiếp; baseline đối chứng naive — lấy giá hôm nay đoán cho ngày mai. Baseline này nghe “lười” nhưng với giá tài sản là đối thủ khó — ai quá nó mới chứng tỏ học được động lượng.

SLIDE 22 — Kết quả & nghịch lý (~75 giây)

Kết quả RMSE trên tập test, đọc bảng: Giá vàng — naive 15.48, PyTorch RNN 15.45 nhỉnh hơn, nhưng Keras 16.42 — tệ hơn cả naive. Amazon — naive 3.14, PyTorch 3.13 xấp xỉ, Keras 3.18 thua.

Nghịch lý quan sát: mạng học sâu huấn luyện hàng chục epoch chỉ ngang, thậm chí thua một thuật toán “lười” lấy giá hôm trước làm dự đoán. Slide sau giải thích bản chất — không phải mạng yếu, là bài toán đặt sai.

SLIDE 23 — Bản chất thất bại: dịch chuyển phân phối (~75 giây)

Vì sao dự báo mức giá thất bại? Vì phân phối train và test KHÔNG giao nhau: tập train giá cao nhất 850 đô — đã học, nội suy tốt; tập test giá THẤP NHẤT đã 1.049 đô — 100 phần trăm dữ liệu test cao hơn giá trị train max — mạng chưa gặp bao giờ, ngoại suy bất lực.

Hai bài học: mạng nơ-ron không ngoại suy được vùng số liệu mới; và giải pháp — không dự đoán mức giá tuyệt đối mà chuyển sang LOG-RETURN r-t bằng $\ln$ của P-t chia P-t trừ 1: thước đo biến động tương đối phân phối ổn định qua các thời kỳ, mạng nội suy lại được.

SLIDE 24 — LSTM (~75 giây)

LSTM 1997 thêm đường truyền thẳng cell state C-t — bằng chuyển trí nhớ dài hạn — không bị cản trở bởi phép nhân ma trận lặp lại: gradient đi qua PHÉP CỘNG và cổng quên nên gần như nguyên vẹn.

Ba cổng, chỉ vào sơ đồ: cổng quên f-t nhân vào băng chuyển — bỏ cái cũ; cổng ghi i-t cộng vào — nạp cái mới đáng chú ý; cổng xuất o-t quyết định trích xuất gì ra trạng thái ẩn. Mạng TỰ HỌC điều gì đáng nhớ, điều gì đáng quên.

SLIDE 25 — GRU (~50 giây)

GRU 2014 tinh giản LSTM: gộp cell state và hidden state thành một trạng thái duy nhất, chỉ hai cổng — cổng cập nhật z-t cân bằng giữ cũ và ghi mới, gộp vai trò quên

cộng ghi; cổng đặt lại r-t quyết định quá khứ kết hợp bao nhiêu với đầu vào hiện tại. Giảm khoảng 25 phần trăm tham số, train nhanh hơn rõ rệt mà hiệu quả thực nghiệm tương đương.

SLIDE 26 — Tổng kết & bức tranh toàn cảnh (~45 giây)

Bảng tiến trình: MLP và 1D-CNN — ép phẳng, trượt cục bộ, mất tính tuần tự dài, chỉ hợp chuỗi ngắn. RNN thuần 1986–1990 — bộ nhớ ẩn h-t, chia sẻ trọng số thời gian, nhưng triệt tiêu gradient, dễ nổ đạo hàm. LSTM GRU 1997–2014 — cơ chế cổng, nhớ dài hạn rất tốt, chuẩn mực time-series. Transformer 2017 — self-attention, xử lý song song tuyệt đối, thống trị LLM.

Thông điệp: Simple RNN đặt nền móng lý thuyết mang tính lịch sử về chia sẻ trọng số thời gian và trạng thái ẩn; hiểu rõ điểm nghẽn toán học của nó chính là chìa khóa làm chủ mọi mô hình học sâu chuỗi hiện đại. Nhóm em xin phép kết thúc, cảm ơn thầy và các bạn đã lắng nghe.

KẾT (~15 giây — trên slide 26)

Đọc thông điệp kết luận trên slide, cảm ơn thầy và các bạn, mời câu hỏi.

Phụ lục — Q&A dự phòng

Câu hỏi	Trả lời ngắn
Vì sao tầng ẩn RNN dùng tanh mà không phải sigmoid hay ReLU?	Tanh tâm 0 nên trí nhớ dương âm cân bằng; sigmoid ra (0,1) luôn dương làm gradient của W cùng dấu mỗi batch — cập nhật đích dắc chậm hội tụ, lại thêm đạo hàm tối đa 0.25 so với 1 của tanh — chuỗi Jacobian tắt nhanh gấp bốn. ReLU không chặn trên: chuẩn h-t cộng dồn qua từng bước nhân U, bùng nổ trạng thái theo độ dài chuỗi. Quan hệ trực tiếp: $\tanh(z) = 2\sigma(2z) - 1$.
LSTM cổng dùng σ nhưng ứng viên $\tilde{c}$ dùng tanh — vì sao?	Cổng là TỈ LỆ giữ lại / phớt lờ — phải thuộc (0,1) như van, sigmoid hợp vai. Ứng viên $\tilde{c}$ là NỘI DUNG trí nhớ mới — cần mang dấu dương âm nên tanh đúng vai. Đổi hai hàm là sai vai trò.
Vì sao không dùng Bidirectional RNN cho dự báo giá?	BiRNN đọc cả hai chiều — tăng ngược nhìn TƯƠNG LAI. Hợp lệ cho NER, dịch máy khi đã có đủ câu; dự báo ngày mai mà cho mạng thấy tương lai là rò rỉ dữ liệu, kết quả ảo. Bài này bắt buộc chiều thuận.
MLP cửa sổ 30 ngày cũng dự báo được — cần RNN làm gì?	MLP cố định đúng L bằng 30 đầu vào, mỗi vị trí trọng số riêng. RNN chia sẻ cùng bộ W, U, V qua mọi thời điểm: tham số không đổi khi L tăng, tổng quát cho chuỗi dài ngắn khác nhau. Với L nhỏ hai bên đều chạy được — RNN thắng ở tính chia sẻ và mở rộng độ dài.
Clipping trị được cả triệt tiêu gradient không?	Không. Bùng nổ và triệt tiêu cùng một cơ chế — tích chuỗi chứa U mũ số bước: chuẩn U lớn hơn 1 thì bùng, nhỏ hơn 1 thì tắt. Clipping chỉ co

	gradient LỚN về ngưỡng — tức trị bùng nổ; gradient đã gần 0 thì không có gì để kéo giãn. Triệt tiêu phải chữa bằng kiến trúc — đường cộng của LSTM, GRU.
Vì sao phải chia theo thời gian và chỉ fit scaler trên train?	Xáo trộn để mẫu tương lai lọt vào train là rò rỉ dữ liệu — validation ảo cao, deploy thất bại. Chuỗi thời gian phải chia chronological 70 15 15; mọi tham số chuẩn hóa — min max hay trung bình độ lệch — chỉ được tính trên train rồi áp cho val test.
Con số 4353 tham số ở đâu ra?	$P = d_h \cdot d_x + d_h^2 + d_y \cdot d_h + d_h + d_y$ với $d_x = 2, d_h = 64, d_y = 1$: W 128, U 4096, V 64, b_h 64, b_y 1 — cộng lại 4353, không chứa T: chuỗi 5 hay 5.000 bước đều dùng đúng bộ này.
Ví dụ forward slide 12 kiểm chứng bằng cách nào?	PyTorch nn.RNN(2,2) + nn.Linear(2,1): nạp W vào weight_ih_l0, U vào weight_hh_l0, V vào trọng số Linear; đặt hai bias của nn.RNN về 0 và bias Linear bằng 0.1 — forward ra $\hat{y}-1$ bằng 0.662, khớp tính tay sai số dưới 10 mũ trừ 6.
Keras RNN thua naive trên tập vàng (16.42 so 15.48) — mạng RNN yếu à?	Không phải kiến trúc yếu mà là bài toán đặt sai: dự báo MỨC GIÁ tuyệt đối khi phân phối test trượt hẳn khỏi train — mọi mạng đều bị kéo về trung bình train. Chuyển sang log-return thì thước đo ổn định và mạng vượt naive; slide 23 đã giải thích cơ chế.
Truncated BPTT là gì? Bài này có cần không?	Cắt lan truyền ngược chỉ k bước gần nhất thay vì cả chuỗi — dùng khi chuỗi hàng nghìn bước để tiết kiệm bộ nhớ. Cửa sổ bài này L bằng 30 nên BPTT trọn chuỗi vừa sức, không cần cắt.